Reactive Tool Manipulation with a Multi-Task State-Conditioned Language-Action Flow Model

Olivia Taylor
Department of Computer Science
Stanford University, United States
otaylor@stanford.edu

Tyler Lum*
Department of Computer Science
Stanford University, United States
tylerlum@stanford.edu

Jeannette Bohg*
Department of Computer Science
Stanford University, United States
bohg@stanford.edu

Abstract

We present a reactive state-conditioned Language-Action model for dexterous tool manipulation that predicts short-horizon tool trajectories from a natural-language instruction and the current state of the tool and the object it acts on. A compact transformer trained with conditional flow matching jointly models four tasks—sweep with a brush, flip with a spatula, pour with a spoon, and hammer a nail. We train it on procedurally generated demonstrations in which the planner repeatedly observes the scene, predicts the next stretch of motion, and lets the object move before observing again, so the policy learns to react as the scene changes. We then refine the pretrained policy in simulation with group relative policy optimization (GRPO) over a mixed batch of all four tasks. Trajectories are expressed in a contact frame anchored to the tool's functional surface, which lets a single policy transfer across tools of different shapes. The predicted poses are handed to the SimToolReal controller, which executes them on a 22-DoF Sharpa five-fingered hand mounted on a 7-DoF KUKA iiwa 14 arm.

1 Introduction

Tool manipulation requires reasoning about functional contact geometry—where a tool meets the world, how it should be oriented, and how both evolve as the object being acted on moves. End-to-end vision-language-action models [2, 8, 11] learn visuomotor policies from large demonstration corpora but require substantial task-specific data and can be difficult to adapt to new tool–object interactions. Modular approaches decouple semantic planning from low-level control [1, 5, 6], but prior language-conditioned *goal* prediction predicts a single static configuration rather than motion that must be revised as the scene evolves.

We address *reactive* tool manipulation: the system observes the current tool pose and object state, predicts a short-horizon trajectory, executes the beginning of it, and then reobserves and replans. This pattern matches how people use tools—sweeping crumbs that scatter, flipping food that may miss the pan, pouring contents that deplete, or hammering a nail that sinks incrementally.

*Advising role.

The high-level trajectory is executed by the SimToolReal controller [6] on a 22-DoF Sharpa five-fingered left hand mounted on a 7-DoF KUKA iiwa 14 arm, so the policy need only specify where and how the tool's working surface should move.

Our contributions are:

1. A **contact-frame trajectory representation**—a sequence of waypoints, each giving a contact point, a contact normal, and an in-plane direction—anchored to the tool's functional surface. Because the policy reasons about where this working surface should contact the world rather than about a specific tool mesh, a single model transfers across tools of different geometries.

2. **Procedural reactive demonstrations** for four tool tasks, in which an analytic expert is repeatedly re-queried from the updated scene state, including scenes where the object sits in a pan that is modeled and placed relative to the object.

3. A **single joint state-conditioned Language-Action model** trained across all four tasks with flow matching and then refined in simulation with multi-task GRPO.

4. A **closed-loop deployment** in which the model replans from live observations and works with SimToolReal to execute the four tasks on a real robot.

2 Related Work

Vision-Language-Action Models. Recent VLAs adapt large pretrained vision-language backbones for robot control by adding action prediction heads [2, 3, 8, 11–13]. π_0 [2] introduced a flow-matching action expert on a frozen VLM; OpenVLA [8] demonstrated competitive open-source fine-tuning on robot datasets; GR00T N1 [11] combines VLM reasoning with a diffusion action module. Unlike these end-to-end image-conditioned policies, our model conditions on a compact symbolic state—the tool contact pose, the object pose, the destination, and the table—together with a language instruction, enabling lightweight deployment without a vision backbone at inference.

Language-Conditioned Manipulation. SayCan [1] grounds language plans in affordance functions; scaffolding approaches [5] use VLMs to decompose dexterous tasks into waypoints. An earlier version of our system predicted a single goal configuration per step; here we instead predict full reactive trajectories that are re-queried from live state.

Flow Matching for Robot Control. Flow matching [9] learns vector fields that transport noise to data along straight-line paths, enabling stable training and fast sampling compared to diffusion-based action models [4]. π_0 [2] applies flow matching to continuous action chunks; we apply it to short tool trajectories represented in a contact frame.

RL Fine-Tuning of Generative Policies. Group relative policy optimization [16] normalizes rewards within sampled groups and updates the policy with a clipped surrogate, avoiding a learned critic. We adapt it to a stochastic flow-matching policy by scoring sampled trajectories in closed-loop simulation.

Tool Manipulation and Simulation. SimToolReal [6] provides an object-centric controller for zero-shot dexterous tool use; our model supplies the high-level trajectory that it tracks on the hand. Procedural and simulation data generation [7, 10] complements real demonstrations; we generate expert reactive demonstrations analytically and use simulation for reinforcement learning and evaluation.

Keypoint and Contact Representations. CLIP [14] grounds text labels in a shared embedding space; we use frozen CLIP encodings for instruction and object labels. Each waypoint defines a right-handed contact triad from the contact normal and in-plane direction, which maps to a tool pose via a fixed tool-specific transform.

3 Problem Formulation

At each replan, the system receives a natural-language instruction ℓ, the current tool contact pose (a contact point $\mathbf{p}_c$, contact normal $\mathbf{n}$, and in-plane direction $\mathbf{s}$ orthogonal to $\mathbf{n}$), the object pose, the destination, and a table reference. The policy predicts a trajectory of K=15 contact-frame waypoints:

$$\hat{\mathbf{Y}} = [\hat{\mathbf{y}}_1, \ldots, \hat{\mathbf{y}}_K], \quad \hat{\mathbf{y}}_i = [\hat{\mathbf{p}}_{c,i};\ \hat{\mathbf{n}}_i;\ \hat{\mathbf{s}}_i] \in \mathbb{R}^9. \tag{1}$$

Each waypoint defines a contact-frame rotation $\mathbf{R}_i = [\hat{\mathbf{s}}_i,\ \hat{\mathbf{n}}_i \times \hat{\mathbf{s}}_i,\ \hat{\mathbf{n}}_i]$ and, with a fixed transform from the contact frame to the tool root, yields a 6-DOF tool goal pose for the controller.

Closed-loop execution. A single controller executes the beginning of the predicted trajectory—either a fixed number of leading waypoints or as many as fit in a fixed time window—then reobserves the tool and object state and queries the model again. Training data mirrors this loop: each scene is a sequence of replanning steps, each pairing the current state with the expert's next short-horizon plan.

4 Method

4.1 Contact-Frame Trajectory Representation

We represent the tool at each waypoint by the contact point on its functional surface, the outward contact normal, and an in-plane direction $\mathbf{s}$ describing the orientation of that surface (for example, the direction the brush sweeps or the way the spatula blade points). Normals and in-plane directions are unit vectors; at post-processing, $\mathbf{s}$ is projected orthogonal to $\mathbf{n}$ and renormalized. Anchoring the trajectory to the working surface—rather than to a particular tool mesh—lets the same representation describe tools of different geometries, since the policy reasons about how the functional surface should contact the world. The representation is compact (nine numbers per waypoint), differentiable for flow matching, and composes directly with SimToolReal's object-centric tracking.

4.2 Model Architecture

The policy is a self-attention transformer with hidden size 512, four layers, and eight heads. Its input is a short sequence of tokens:

`[instruction, tool, object, destination, table, noisy trajectory]`,

where object/destination tokens are omitted when not relevant to a task. Each semantic label is encoded by a frozen CLIP ViT-B/32 text encoder [14] and projected to the model dimension; each position passes through a small MLP on normalized coordinates and is added to its label embedding. The final token carries the current noisy trajectory. Each transformer block applies pre-norm self-attention followed by an MLP, with an adaptive normalization conditioned on the flow-matching timestep. The hidden state of the trajectory token is projected to the predicted flow velocity.

4.3 Flow-Matching Objective and Inference

Training uses conditional flow matching [9]. For an example with normalized target trajectory $\mathbf{x}_1$,

$$\mathbf{x}_0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad t \sim \mathcal{U}(0, 1), \tag{2}$$

$$\mathbf{x}_t = (1 - t)\mathbf{x}_0 + t\mathbf{x}_1, \quad \mathbf{v}^* = \mathbf{x}_1 - \mathbf{x}_0, \tag{3}$$

and the model minimizes $\|\hat{\mathbf{v}}_\theta(\mathbf{x}_t, t, \text{cond}) - \mathbf{v}^*\|^2$. Only contact positions are affine-normalized; normals and in-plane directions are trained as raw 3-vectors and normalized after integration. At inference we integrate the learned ODE with 30 Euler steps, draw four independent samples, and average their final trajectories.

4.4 Procedural Reactive Demonstrations

We generate demonstrations for four tasks with analytic expert planners:

- **Sweep with brush**: push loose debris across the table into a receptacle.
- **Flip with spatula**: slide under a flat object, lift, and invert it.
- **Pour with spoon**: scoop the object and roll to pour it out.
- **Hammer nail**: strike a nail head until it reaches a target depth.

For flip and pour, the object starts inside a pan that is modeled in the scene and placed relative to the object (we do not model a separate serving dish). Starting the object against the pan wall gives the tool something to push against, which makes these tasks tractable; without it, sliding a spatula under a flat object or scooping with a spoon is very difficult.

Reactive demonstrations. Each scene is rolled out as a closed loop. At every step the expert plans a 15-waypoint trajectory, executes the first few waypoints, and lets the scene update before planning again. Between steps we perturb the scene—jitter or knock the object, drop a scoop, fail a flip, or land a weak hammer hit—so the expert must recover, and the resulting demonstrations teach the policy to react rather than to follow a fixed script. Each step yields one training example that pairs the current observation with the expert's next plan.

Natural-language instructions. Each scene is specified by a free-form natural-language instruction sampled once and reused across that scene's replanning steps. To encourage generalization across phrasing rather than memorizing a fixed command, we generate a large and diverse instruction set per task by combining many templated sentence structures with varied synonyms for the tool, the object, and the destination (for example, "sweep the crumbs into the dustpan" versus "brush the debris toward the tray"). This exposes the model to a wide range of wordings for the same underlying task.

Scale. The four tasks are balanced at the example level: the per-task data caps are chosen so that each task contributes a comparable number of training examples (on the order of 2×10^5 each, roughly 10^6 in total), even though brush sweep is spread over more shards. Labels are produced analytically; simulation is used only for reinforcement learning and evaluation.

4.5 Joint Multi-Task Pretraining

We pretrain on all four tasks jointly, concatenating their demonstrations and shuffling across tasks, with a global per-axis normalization of contact positions computed over the combined data. Training runs for ten epochs with batch size 128 and AdamW (learning rate 5×10^{-4}, brief warmup then cosine decay). This checkpoint is the starting point for reinforcement learning.

4.6 GRPO Simulation Fine-Tuning

We refine the policy with group relative policy optimization [15, 16]. For each scene the policy samples several trajectories by integrating the flow with injected noise and recording the probability of the sampled path. Each trajectory is executed in closed-loop simulation and assigned a scalar task reward. Advantages are normalized within each scene's group of samples,

$$A_i = \frac{r_i - \bar{r}}{\sigma_r + \epsilon}, \tag{4}$$

and the policy is updated with a clipped likelihood-ratio surrogate plus a penalty that keeps it close to the pretrained model. A single training run mixes all four tasks, sampling an equal number of scenes from each per iteration, and checkpoints are saved periodically.

Per-task rewards.

- **Brush**: reward for the swept object reaching the goal region, combined with a term for how well the hand tracked the requested trajectory.
- **Flip**: reward for the object ending inverted and settled near the table, with partial credit for inversion progress.
- **Pour**: reward for the poured object reaching the destination and settling, with partial credit for progress toward it.
- **Hammer**: reward for how far the nail is driven toward its target depth, with full credit on reaching it.

5 Closed-Loop Deployment

At deployment a single closed-loop controller handles all four tasks. It observes the current tool and object pose, predicts a 15-waypoint trajectory, and converts each contact-frame waypoint into a tool goal pose, which it hands to SimToolReal for execution on the hand. Tool and object poses are estimated with FoundationPose [17]—the same pose estimator SimToolReal uses for tool tracking—so perception integrates cleanly across the two systems with no additional calibration. It then executes the beginning of the plan—either a fixed number of waypoints or as many as fit in a short time window—reobserves, and replans, repeating until the object reaches its destination. The model operates in its own world frame, so observations are transformed in from the robot frame

Table 1: Mixed-task GRPO training metrics for the joint four-task run, at iterations 10 and 40. Reward is the mean over the mixed four-task batch (arbitrary units, with per-episode success scoring up to ≈ 1.5); tracking is the low-level trajectory-following fraction in $[0, 1]$. Per-task success rates were not logged separately during this run.

Metric	Iter 10	Iter 40
Mean reward (mixed tasks)	0.37	0.41
Mean tracking fraction	0.99	1.00

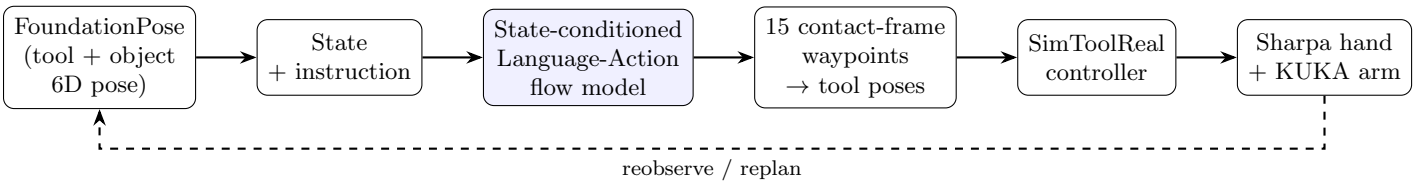

Figure 1: System overview. FoundationPose estimates tool and object poses; together with the instruction these form the symbolic state for the state-conditioned Language-Action model, which predicts a contact-frame trajectory. Waypoints are converted to tool goal poses and tracked by SimToolReal on the Sharpa hand and KUKA arm. The loop closes by reobserving and replanning.

and predicted poses are transformed back out before execution. The same package drives both the real robot and an interactive visualization used for development, in which the predicted trajectory and tool poses can be inspected before running on hardware.

6 Experiments

Evaluation setup. Simulation evaluation uses the same closed-loop protocol as reinforcement learning, with per-task success defined as in Section 4.6 (object in the goal region, object inverted, object poured, or nail driven to depth). On the real robot, the closed-loop controller drives SimToolReal and success is measured by task completion.

Qualitative results. The jointly pretrained model produces coherent sweep arcs, flip trajectories that approach the pan, pour paths that empty into the pan, and hammer strikes that descend toward the nail. During mixed-task GRPO the mean reward rises from 0.09 at initialization to a band around 0.35–0.45 within the first few iterations, while low-level tracking stays near-saturated (≈ 0.99); Table 1 reports two checkpoints along this trajectory. On the real robot, closed-loop replanning yields continuous tool motion across replans.

7 Limitations and Future Work

Model. The model does not condition on images, so visual obstacles and fine geometry must be inferred from sparse state. Training labels come from analytic experts rather than human or simulated rollouts, which may not cover every failure mode, and the reinforcement-learning rewards are task-specific and bounded by simulation. Robot time was limited, so although the model is trained on all four tasks, our real-robot rollouts cover only the blue-brush and red-brush sweep tasks and claw-hammer nailing; the spatula-flip and spoon-pour tasks were exercised in simulation and in the interactive visualization but not yet on hardware.

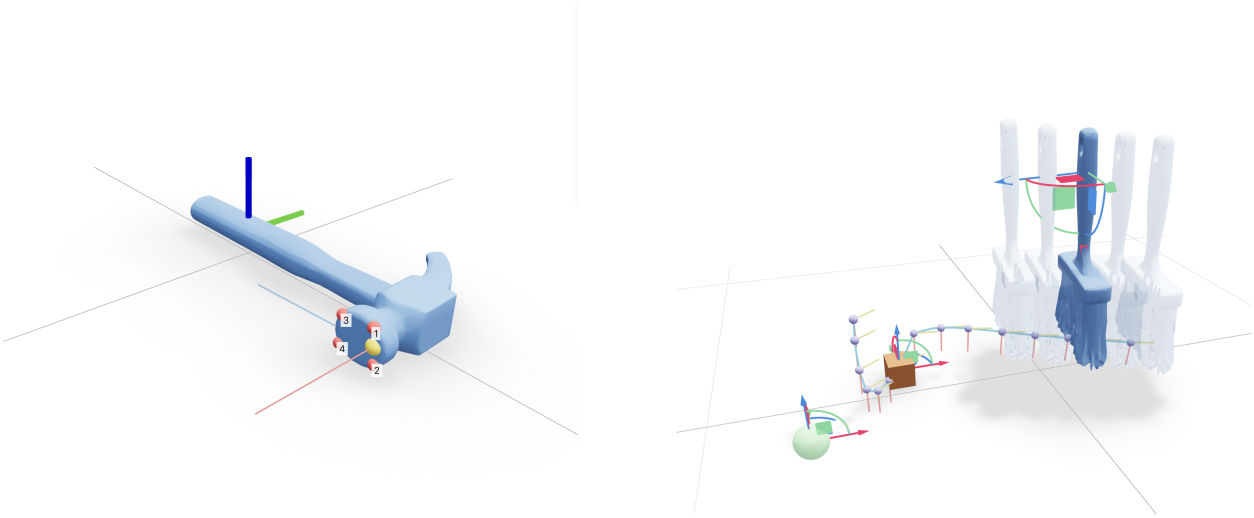

Figure 2: Left: the contact-frame definition for the hammer—a contact point with its normal and in-plane direction anchored to the tool's functional surface. Right: a predicted brush-sweep trajectory of contact-frame waypoints.

Perception and execution failures. Because both our system and SimToolReal rely on FoundationPose for pose estimation, tracking errors propagate end-to-end: the SimToolReal policy would occasionally lose the object during execution, after which the predicted goals no longer matched reality.

Orientation behavior. On some replans the policy predicts an orientation flipped by roughly 180° about the contact normal—it negates the in-plane direction even though the contact point and normal are unchanged—producing a discontinuous jump in the tool's goal orientation. When not fully corrected, this jump made the arm try to move too fast to catch up and sometimes drop the object. We mitigate it at deployment by aligning each new plan's orientation to the previous one and choosing the in-plane direction closest to it, without otherwise changing the contact frame, but this is a workaround for behavior the policy should not exhibit in the first place. Relatedly, the policy tended to prefer a particular tool orientation and would force the trajectory to follow it even when another orientation would have worked equally well (for example, sweeping the brush from the opposite direction).

Future work. We expect the orientation preferences and discontinuities to respond to more aggressive domain randomization during training and to a penalty for extra steps in simulation (discouraging the policy from committing to a single orientation and from taking redundant motions), alongside removing the flips at their source through orientation-aware training, image conditioning, and larger-scale multi-task reinforcement learning.

8 Conclusion

We presented a reactive multi-task state-conditioned Language-Action model for dexterous tool manipulation that predicts contact-frame tool trajectories from language and the current scene state, trained on procedural reactive demonstrations across four tasks, refined with multi-task GRPO in simulation, and deployed in a closed loop that works with SimToolReal to execute on a

dexterous hand. The system moves from single-goal prediction to continuous closed-loop trajectory generation, with a tool-agnostic representation that bridges learning and low-level execution.

Acknowledgments

We thank the SimToolReal team for the low-level execution stack and simulation environments.

References

[1] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.

[2] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Max Equi, Chelsea Finn, Niccolò Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[3] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.

[4] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Ben Burchfiel, Shuran Song, and Russ Tedrake. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 2024.

[5] Victor de Bakker, Joey Hejna, Tyler G. W. Lum, Onur Celik, Anja Taranovic, Dominik Blessing, Gerhard Neumann, Jeannette Bohg, and Dorsa Sadigh. Scaffolding dexterous manipulation with vision-language models. *arXiv preprint arXiv:2506.19212*, 2025.

[6] Kushal Kedia, Tyler G. W. Lum, Jeannette Bohg, and C. Karen Liu. Simtoolreal: An object-centric policy for zero-shot dexterous tool manipulation. *arXiv preprint arXiv:2602.16863*, 2026.

[7] Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kevin Ellis, et al. Droid: A large-scale in-the-wild robot manipulation dataset. *arXiv preprint arXiv:2403.12945*, 2024.

[8] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Tony Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[9] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023.

[10] Mayank Mittal, Philipp Roth, Joseph Tigue, Alexander Richard, Oliver Zhang, Peng Du, Alvaro Serrano-Munoz, Xinyu Yao, Nicolas Zurbrügg, Nikita Rudin, et al. Isaac lab:

A gpu-accelerated simulation framework for multi-modal robot learning. *arXiv preprint arXiv:2511.04831*, 2025.

[11] NVIDIA, Filip Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingyu Da, Rui Ding, Linxi Fan, Yuxin Fang, Dieter Fox, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.

[12] Physical Intelligence. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.

[13] Physical Intelligence. $\pi_{0.7}$: A steerable generalist robotic foundation model with emergent capabilities. *arXiv preprint arXiv:2604.15483*, 2026.

[14] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning*, pages 8748–8763. PMLR, 2021.

[15] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[16] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[17] Bowen Wen, Wei Yang, Jan Kautz, and Stan Birchfield. Foundationpose: Unified 6d pose estimation and tracking of novel objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.